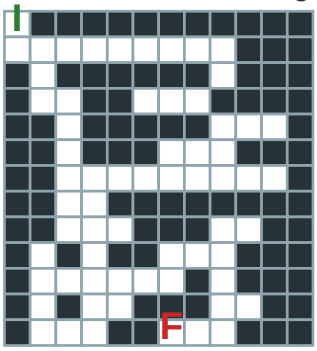
Busca em Labirintos

DFS, BFS e busca gulosa — código explicado visualmente

Labirinto usado no código



Legenda: I início **F** saída **#** parede ***** caminho encontrado

Objetivo: sair de **I** e chegar a **F**, andando apenas para cima, direita, baixo ou esquerda. As paredes # não podem ser atravessadas.

O programa transforma cada casa livre em um ponto do grafo. Duas casas ficam ligadas quando é possível andar diretamente entre elas.

O que a atividade pede

A explicação deve responder: (a) por que a BFS encontrou o menor caminho e a DFS não; (b) em que momento a gulosa foi enganada por uma parede; e (c) como o caminho foi reconstruído a partir das casas visitadas.

1. Lendo o labirinto e criando os movimentos

O labirinto é representado por uma lista de strings. Cada string é uma linha da grade e cada caractere é uma coluna.

```
labirinto = [
    "I###########",
    ".........###",
    ...,
    "#...##F..###",
]
```

Para acessar uma casa, o código usa `lab[coluna][linha]`. Apesar dos nomes das variáveis, o primeiro índice representa a linha da grade e o segundo representa a posição dentro da string.

A função `livre` verifica duas coisas: se a coordenada está dentro da grade e se o caractere não é `#`.

```
DIRECOES = [(-1, 0), (0, 1), (1, 0), (0, -1)]

def livre(lab, coluna, linha):
    dentro = 0 <= coluna < len(lab)
    and 0 <= linha < len(lab[0])
    return dentro and lab[coluna][linha] != "#"
```

Como o código se move: **(-1, 0)** sobe uma linha; **(0, 1)** vai para a direita; **(1, 0)** desce uma linha; **(0, -1)** vai para a esquerda. A função nunca permite movimento diagonal.

O grafo guarda os vizinhos possíveis de cada casa:

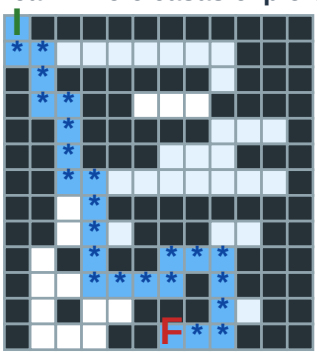
```
grafo[(coluna, linha)] = [vizinhos livres]
```

Assim, uma busca não precisa testar o desenho inteiro a cada passo: ela consulta no grafo para onde pode ir a partir da posição atual.

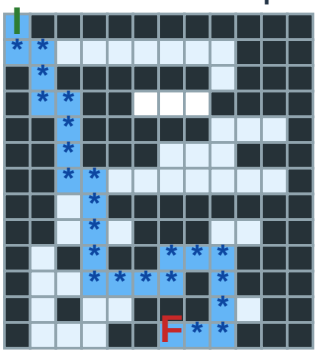
2. Visualização dos caminhos

As três buscas encontraram 24 movimentos neste labirinto. A cor azul clara mostra casas exploradas; o azul mais forte e o símbolo * mostram o caminho retornado.

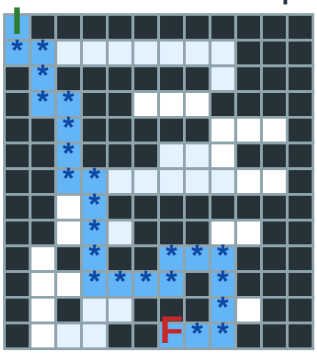
DFS: caminho e casas exploradas



BFS: caminho e casas exploradas



Gulosa: caminho e casas exploradas



Neste exemplo, os três caminhos coincidiram. Isso acontece por causa do formato específico do labirinto; não significa que DFS e gulosa sempre encontrem o menor caminho.

3. Como cada busca pensa

DFS — busca em profundidade: usa uma pilha. Ela escolhe um caminho e tenta seguir o mais longe possível. Se encontrar a saída, para. Como não compara primeiro todos os caminhos curtos, pode encontrar um caminho maior.

```
pilha = [inicio]
atual = pilha.pop()
pilha.append(vizinho)
```

Visualmente, a DFS parece uma pessoa entrando em um corredor e seguindo até o fim. É boa quando basta encontrar qualquer solução e quando o espaço de memória precisa ser menor que o da BFS. Não é a melhor escolha quando o menor caminho é obrigatório.

BFS — busca em largura: usa uma fila. Ela visita primeiro todas as casas a 1 movimento do início, depois as casas a 2 movimentos, depois a 3 e assim por diante. Por isso, em um labirinto em que cada movimento custa 1, o primeiro caminho que chega à saída é o menor.

```
fila = deque([inicio])
atual = fila.popleft()
fila.append(vizinho)
```

Visualmente, a BFS forma ondas saindo do início. É indicada para mapas sem pesos, quando o objetivo é garantir o menor número de movimentos.

Busca gulosa: usa uma fila de prioridade e escolhe a casa que parece mais próxima da saída. No código, a estimativa é a distância Manhattan: **abs(linha atual** – **linha destino) + abs(coluna atual** – **coluna destino)**.

```
prioridade = heuristica(vizinho, saida)
heappush(fronteira, (prioridade, vizinho))
_, atual = heappop(fronteira)
```

Ela é indicada quando uma direção aproximada até o objetivo ajuda a explorar menos casas e uma solução rápida é mais importante que a garantia de ser a melhor.

O que é a heurística?

Heurística é uma estimativa usada para orientar a busca. Ela não precisa ser o custo real completo; serve como uma dica de qual opção parece promissora.

```
def heuristica(posicao, destino):
    return abs(posicao[0] - destino[0])
    + abs(posicao[1] - destino[1])
```

A distância Manhattan soma quantas linhas e colunas ainda separam duas casas, como se não existissem paredes. Por isso é apenas uma estimativa: uma parede pode obrigar a fazer uma volta.

Exemplo: se a posição atual é (2, 3) e a saída é (5, 7), a estimativa é **|2–5| + |3–7| = 3 + 4 = 7**. A busca gulosa prefere a alternativa com menor valor estimado.

A gulosa pode ser enganada por uma parede porque olha apenas para a distância estimada. Uma casa pode parecer muito perto da saída, mas a parede obrigar o caminho real a dar uma volta. Ela então explora outras casas até encontrar uma passagem.

Comparação real deste labirinto

Busca	Estrutura	Movimentos	Casas visitadas	Menor caminho garantido?

DFS	Pilha	24	50	Não
BFS	Fila	24	61	Sim, com custos iguais
Gulosa	Fila de prioridade	24	45	Não

4. Como o caminho é reconstruído

Enquanto visita as casas, cada busca registra de onde chegou. Isso acontece nesta linha:

```
anteriores[vizinho] = atual
```

Exemplo simples:

Casa	Veio de
(0, 0) — início	None
(1, 0)	(0, 0)
(1, 1)	(1, 0)
(2, 1)	(1, 1)
(3, 1)	(2, 1)

Quando a saída é encontrada, a função `reconstruir_caminho` começa na saída e consulta repetidamente `anteriores`. Assim ela volta até o início. Como essa ordem fica de trás para frente, `caminho[::-1]` inverte a lista e coloca o caminho na ordem correta.

Passo a passo:

1. A busca encontra uma casa nova.
2. Salva o antecessor dessa casa.
3. Chega à saída.
4. Volta da saída consultando os antecessores.
5. Inverte a sequência e desenha cada casa do caminho com *.

5. Quando usar cada busca

Use DFS quando qualquer caminho serve, quando a memória é limitada ou quando você quer explorar profundamente uma estrutura.

Use BFS quando cada movimento tem o mesmo custo e você precisa do menor número de movimentos.

Use gulosa quando uma estimativa até o objetivo pode acelerar a procura e você aceita que o caminho talvez não seja o melhor.

Resposta pronta para apresentar

A BFS encontrou o menor caminho porque explora o labirinto por camadas: primeiro as casas mais próximas do início, depois as mais distantes. A DFS não garante o menor caminho porque segue profundamente uma direção e para assim que encontra uma saída. A busca gulosa pode ser enganada quando uma parede bloqueia a direção que parece mais curta; nesse caso, ela precisa contornar a parede. O caminho é reconstruído usando o dicionário `anteriores`, que guarda de qual casa cada posição foi alcançada. Depois, a sequência é invertida para ficar do início até a saída.

Observação: neste labirinto, as três buscas encontraram 24 movimentos, mas visitaram quantidades diferentes de casas. O resultado da BFS é o único que mantém a garantia do menor caminho para movimentos de mesmo custo.